

Submit your assignment here: <https://forms.gle/cWdubSjx7teKoQDk9>

CANDIDATE BRIEF

Talk to Government Data

A natural-language analyst over open public data

ROLE

Applied GenAI Intern

PLATFORM

Google Colab + a simple connected UI

FORMAT

Build + short design note

This is a screening exercise, not a take-home project. We are looking for sound judgment and solid working fundamentals over polish. If you find yourself over-engineering, stop and write down what you *would* do instead, then move on.

1 Context

Governments sit on enormous amounts of **structured open data** — crop yields, road accidents, air quality, school enrolment, budget allocations, vehicle registrations. It is all public, but it lives in CSVs and portals that ordinary citizens, and most officials, cannot actually query. The question *"how did paddy production in my state change over the last five years?"* has a precise answer sitting in a spreadsheet, with no easy way to ask it.

Your task: build a small tool that lets a **non-technical person ask a question in plain English** and get back a correct, data-backed answer — a number or trend, a chart, and a short note on how it was computed.

THE CORE RULE — THE WHOLE POINT OF THE EXERCISE

Every figure in an answer must be computed from the actual dataset — never generated by the language model from memory. A confident, made-up statistic about a government scheme is worse than "I can't answer that from this data."

2 Pick a real dataset

Choose **one** real open dataset from a government source. Use something with enough structure to ask interesting questions (multiple columns, a time or geography dimension), but small enough to handle in Colab (a single CSV / XLSX, ideally under ~100 MB).

Where to get it

data.gov.in — Open Government Data Platform India (CSV / XLS / JSON, with APIs) data.gov.in

AIKosh — IndiaAI Datasets Platform: agriculture, health, language, and more aikosh.indiaai.gov.in

Any other official portal — data.gov.uk, a state portal, or a specific ministry data release

Example domains and questions

Pick one, or bring your own — these exist only to get you started.

Domain	Typical dataset	Example questions a user might ask
Agriculture	Crop production by state/year; mandi prices (Agmarknet)	"Which state produced the most wheat in 2022?" · "How has onion price volatility changed over 3 years?"
Road safety	Road accident statistics (MoRTH); vehicle registrations (Vahan)	"Which states have the highest fatal-accident rate per vehicle?" · "Did two-wheeler accidents rise after 2020?"
Public health	Immunization coverage; hospital counts; NFHS indicators	"Compare child immunization rates across these 5 states." · "Where is hospital-bed availability lowest?"
Air & environment	CPCB air-quality (AQI) readings; water quality	"Which city had the worst average PM2.5 last year?" · "How does AQI vary by season in Delhi?"
Education	School enrolment / UDISE+; literacy	"What is the student-teacher ratio trend by district?" · "Which states improved enrolment the most?"
Public finance	Union / state budget allocations and expenditure	"How did the health-budget share change over 5 years?" · "Top 5 ministries by allocation growth."
Crime / safety	NCRB crime statistics	"Which categories of crime rose fastest nationally?"

State clearly in your submission **which dataset you chose and a direct link to it**.

3 What to build

A tool that, given a plain-English question, will:

1. **Understand the question** and work out what operation it implies — filter, aggregate, group, compare, trend over time, top-N, and so on.
2. **Compute the answer from the real data** — the model should produce a query or code (e.g. pandas / SQL) that is *actually executed* against the dataset; the answer is built from the result, not from the model's prior knowledge.
3. **Return** a clear natural-language answer, the relevant **number(s)**, and a **chart** where it makes sense (a trend line, a bar comparison, etc.).
4. **Show provenance** — at minimum, surface the query/code that ran, or a plain-English description of the steps, so the answer is auditable.
5. **Refuse gracefully when out of scope** — if a question cannot be answered from the chosen dataset (wrong topic, missing column, missing year), say so clearly instead of inventing a figure.
6. Ship with a small **evaluation set** of 5-8 questions you write yourself, including at least one **out-of-scope question that should be refused**, and at least one where you have **manually verified the correct number** so you can confirm the system is not hallucinating.

You choose the stack: any framework (LangChain, LlamaIndex, or hand-rolled), any LLM (Claude API, an open model, another provider), and pandas or DuckDB / SQLite for the data layer. We care about *why* you chose what you chose, not which logo you picked.

A DESIGN FORK WORTH THINKING ABOUT

Letting an LLM emit raw Python and running `exec()` on it is the quick path — but you are now executing model-generated code, which is a real risk. A safer pattern is to constrain the model to emit a **structured** operation (e.g. JSON describing the filter / aggregation) that **your** trusted code executes. Either approach is acceptable; tell us which you chose and why in your design note.

4 Required architecture: Colab + a connected UI

Everything must run from a **single Google Colab notebook**, so we can reproduce it with "Run all." On top of that, wire up a **simple web UI that connects to your pipeline** — not just notebook cells. Two acceptable paths:

- **Recommended — Gradio (or Streamlit) in Colab.** Launch a `gradio` interface from the notebook with `share=True` to get a public link. The fastest way to get a real UI talking to your backend without leaving Colab. The share link is temporary, so capture a screenshot or a short screen recording of it working.
- **Advanced (optional) — API + frontend.** Run a FastAPI / Flask endpoint in Colab, expose it via `ngrok`, and call it from a minimal HTML/JS or React page. More production-shaped, but more fragile in Colab — only if you are comfortable.

Either way the point is the same: **a UI that sends a question to a backend which loads the data, runs a real computation, and returns the result.** A cell that just prints a hardcoded answer does not count.

Housekeeping we notice: do not hardcode API keys in the notebook — use Colab *Secrets* (the key icon in the sidebar) or an environment variable. A leaked key in a submitted notebook is an automatic red flag in this domain.

5 The design note

Working code is half the signal. In a `README.md` or a Markdown cell at the top of the notebook, answer:

1. **Stack & decisions.** What did you pick (LLM, data layer, framework) and why? Did you run model-generated code directly or constrain it to a structured query — and what is the trade-off? What would you do with more room?
2. **Correctness & trust.** How do you keep the model from fabricating numbers? How would you *prove* to a sceptical government officer that an answer is right?
3. **Government deployment.** This pattern executes computations over official, possibly sensitive data. How would you handle **data residency, audit trails, access control, and the security risk of executing generated code**? What changes between this prototype and something a public body could run?
4. **Scaling.** What breaks when the data goes from one CSV to hundreds of tables and millions of rows? Name the first two or three things you would fix.
5. **Validation.** How would you test this with non-technical stakeholders? What failures would you watch for?
6. **Honest limitations.** Where does your build fall down today? We would rather see clear-eyed awareness than a hidden bug.

6 Stretch goals — optional, pick at most one

- Support **two related datasets** and a question that needs joining or comparing them.
- Add an **agentic step** — the model decides whether it *can* answer, picks the right column(s), or self-corrects when its first query errors out.
- Add a **confidence signal** and route low-confidence answers to a "needs human review" state.
- A genuinely nice **React / Next.js** frontend instead of Gradio.

7 Deliverables

1. **Colab notebook link** (sharing set to "Anyone with the link can view"), with output cells already run and saved.
2. **Proof of the UI working** — a screenshot or short screen recording showing a question answered (number + chart), and the out-of-scope refusal in action.
3. **The design note** (Section 5).
4. **Your evaluation set + results** — the 5-8 questions, expected vs. actual, including the trap question and at least one hand-verified number.
5. (*Nice to have*) a short "decisions log" of trade-offs and where you got stuck.

8 Ground rules

- **Using AI assistants (Claude, ChatGPT, Copilot, etc.) is allowed and encouraged** — that is the job. But you must understand and be able to explain every line you submit; we may ask you to walk through it.
- **Favour a clean, working core over breadth.** If something is large in scope, describe it in the design note rather than building all of it.
- **Scope is intentionally under-specified.** Making reasonable assumptions and stating them is part of what we assess.

9 How we'll evaluate

So you know where to focus, in rough priority order:

1. **Does it work end-to-end** — question in via the UI, correct number + chart out, computed from real data.
2. **Numbers are real, not hallucinated** — answers trace to an actual computation; the out-of-scope question is refused.
3. **Data handling** — sensible loading and cleaning of messy real-world government data (types, missing values, inconsistent labels).
4. **Design thinking** — the correctness, security (executing generated code), compliance, and scaling answers in your note.
5. **Communication** — can we follow your decisions and your honesty about limitations.

A simple, working, well-reasoned submission beats an ambitious broken one. Good luck.